

Clase 05 (04/21/2026)

1. Revisión clase 4

- *R.E.P.*'s, entorno y *Workspace*
- Paquetes de ROS
- Uso de nombres:
 - de paquete: estructura de archivos, `package.xml`, `setup.cfg`, `setup.py`, línea de comando
 - de archivo: estructura de archivos y `setup.py`
 - de ejecutable: `setup.py` y línea de comando
 - de nodo: código y línea de comando
- Programación de nodo (`rclpy`)
- Funciones *callback*
- Dependencias y compilación

2. Programación Orientada a Objetos

- Tres formas de escribir código en ROS
 - Código **in-line**: Todo contenido en la función `main`. (Ejemplo *publisher*)
 - Funciones locales: Las funciones se separan del `main`. Es necesario "compartir" datos a través de variables *globales*.
 - Orientado a objetos: Utilizando clases. Código más ordenado y legible.
- Creación de una clase a partir de `rclpy.node.Node`
- Constructor y atributos
- Uso de métodos de la clase como *callbacks*

3. `rclpy`: Parámetros

Valores de configuración que pueden ser modificados por el usuario sin necesidad de recompilar el paquete. En los nodos se accede a esta utilidad a través de `rclpy`

- Antes de hacer uso de un parámetro siempre es necesario declararlo
- El tipo se infiere a partir del valor por defecto
- *Callback* de modificación de parámetros (`ParameterEventHandler(...)`)
- Modificar y leer parámetro desde consola. Parámetros como argumentos de ejecución.

4. Logging

5. Laboratorio

5.1. Modificación del código de nodos

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Publicador(Node):
    def __init__(self):
        super().__init__('publicador') # Nombre del nodo
        # 2.1 Creación de publisher
        self.pub = self.create_publisher(String, 'topic', 10)
        # 2.2 Creación de mensaje
        self.msg = String()

        # Cada medio segundo publicar un mensaje
        timer_period = 0.5
        # 2.4 Creación del timer
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    # 2.3 Programación de función de callback
    def timer_callback(self):
        # Rellenar el contenido del campo 'data' del mensaje
        msg.data = 'Mensaje de prueba %d' % self.i
        # Publicar el mensaje
        self.pub.publish(msg)

        # Escribir en la terminal a modo de debug
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1

def main(args=None):
    # 1. Inicialización
    rclpy.init(args=args)

    # 2. Creación de nodo
    node = Publicador()

    # 3. Procesamiento de callback
    rclpy.spin(node)

    # Destruir el nodo
    node.destroy_node()
    # 4. Finalización
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

++ Realizar la modificación del nodo subscriber

5.2. Parámetros en rclpy

Declarar parámetros

```
self.declare_parameter(<nombre>)
self.declare_parameter(<nombre>,<valor>,[<descriptor>])
```

Obtener el valor de un parámetro

```
param = self.get_parameter(<nombre>)
val = param.value
```

Establecer el valor de un parámetro

```
# Crear parámetro
param = rclpy.parameter.Parameter(<nombre>, <tipo>, <valor>)
# Establecer el valor
self.set_parameters([param])
```

5.3. Gestión de parámetros por consola

Establecer el valor de un parámetro

```
ros2 param set <nombre_nodo> <nombre_parametro> <valor>
```

Establecer parámetros a través de un archivo de configuración

```
ros2 param load <nombre_nodo> <nombre_archivo>
```

Establecer parámetros al momento de ejecutar un nodo

```
ros2 run <nombre_paquete> <nombre_ejecutable>
--ros-args -p <nombre_parametro>:=<valor>
```

Ídem anterior pero a partir de un archivo de configuración

```
ros2 run <nombre_paquete> <nombre_ejecutable>
--ros-args --params-file <nombre_archivo>
```

5.4. *Logging*

Generar un mensaje de *log*

```
self.get_logger().{debug,info,warning,error,fatal}(<mensaje_de_log>')
```

Generar un mensaje de *log* por única vez

```
self.get_logger().{debug,info,warning,error,fatal}(<mensaje_de_log>',
once=True)
```

Período mínimo entre mensajes de *log*

```
self.get_logger().{debug,info,warning,error,fatal}(<mensaje_de_log>',
throttle_duration_sec=<period>)
```

Ignorar la primera vez y enviar los mensajes siguientes

```
self.get_logger().{debug,info,warning,error,fatal}(<mensaje_de_log>',
skip_first=True)
```

Cambiar el nivel de severidad al momento de ejecutar un nodo

```
ros2 run <nombre_paquete> <nombre_ejecutable>
--ros-args --log-level {DEBUG|INFO|WARN|ERROR|FATAL}
```